

Reviewer Pack — Minimal Root Multiplicity Index and Communication Complexity...

Entry f6a885161058 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 15:48:46 UTC

Minimal Root Multiplicity Index and Communication Complexity Rank Variance

Entry ID: f6a885161058 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 15:48:46 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Root Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For every boolean function f with n variables, the minimal root multiplicity index of its characteristic polynomial is linearly correlated with the communication complexity rank variance of f , such that the ratio of their logarithms is $O(\log n)$, i.e., $\log(\sqrt[n]I(f)) = \Theta(\log(\kappa(f)))$

Rationale (proposer's reasoning):

Root theory provides a way to analyze the algebraic structure of boolean functions through their characteristic polynomials. Communication complexity rank variance measures the complexity of communicating values about a function. A link between these two areas could reveal underlying algebraic properties that affect communication complexity.

Taxonomy category: root_theory_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc7193e49447d217

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

(prereg LLM failed:)

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "root multiplicity index" AND "communication complexity rank variance" - "characteristic polynomial" AND boolean function AND communication complexity" - "logarithmic correlation" AND root theory AND matrix rank variance

Top relevant hits considered: - [http://arxiv.org/abs/1609.00027v3] The Riemann zeta function and Gaussian multiplicative chaos: statistics on the critical line - [http://arxiv.org/abs/2411.07356v1] Dualities for characteristic polynomial averages of complex symmetric and self dual non-Hermitian random matrices - [http://arxiv.org/abs/nlin/0209056v3] Saddle points in the chaotic analytic function and Ginibre characteristic polynomial - [http://arxiv.org/abs/1806.07549v1] Maximum of the characteristic polynomial for a random permutation matrix - [http://arxiv.org/abs/1704.02186v1] Non-scalar operators for the Potts model in arbitrary dimension - [http://arxiv.org/abs/1509.07269v2] Testing in high-dimensional spiked models

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=35.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot in column i
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        # Swap rows
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below pivot
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n + 1):
                if k == i:
                    A[j][k] = 0
                else:
                    A[j][k] -= factor * A[i][k]
    return A

def characteristic_polynomial(f):
```

```

n = len(f)
A = [[0] * (n+1) for _ in range(n)]
for i in range(n):
    for j in range(n):
        if f[i][j]:
            A[j][i] = 1
A = gaussian_elimination(A)

# Calculate the determinant of the augmented matrix
det = 1
for i in range(n):
    det *= A[i][i]
return det

def communication_complexity_rank_variance(f):
    n = len(f)
    rank = sum(1 for row in f if any(row))
    return rank * (n - rank)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    instances_tested = 30
    n_max = n

    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        # Generate a random boolean function f with n variables
        f = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

        # Compute the characteristic polynomial of f
        det = characteristic_polynomial(f)

        # Compute the communication complexity rank variance  $\kappa(f)$  for f
        kappa_f = communication_complexity_rank_variance(f)

        if kappa_f == 0:
            continue

        # Calculate  $\log(\sqrt{[n]I(f)})$  and  $\log(\kappa(f))$ 
        metric_value = math.log(math.sqrt(abs(det))) - math.log(kappa_f)

        total_metric_value += metric_value

    mean_metric_value = total_metric_value / instances_tested
    support_fraction = 1.0 if conjecture_holds else 0.0

    return {
        "metric_name": "log(sqrt(I(f)) / kappa(f))",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
    }

```


9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test only uses $n = 40$ and tests 30 instances, which is too small to draw a robust conclusion about the conjecture. The metric value being zero could be due to random chance or an underlying issue with the implementation. | next: Increase the number of tested instances and the range of n to better assess the validity of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13798
2	propose	ollama_remote	glm4:latest	0	0	12669
3	propose	ollama_remote	glm4:latest	0	0	12281
4	preregistration	?	?	0	0	15147
5	novelty	ollama_remote	glm4:latest	0	0	13802
6	novelty	ollama_remote	glm4:latest	0	0	9945
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47854
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11850
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48840
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16491
11	critic	ollama_remote	glm4:latest	0	0	56674
12	judge	ollama_remote	glm4:latest	0	0	10164

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 269515 ms total latency. Provider mix: {'ollama_remote': 11, '?': 1}

(full prompt+response transcripts available in `research/audit/f6a885161058.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f6a885161058.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f6a885161058.tar.gz` (if generated)